KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 1 ⋄ poznámky k přednášce

9. Lexikální uzávěry

verze z 20. listopadu 2024

1 Co víme o uživatelsky definovaných funkcích

Na minulé přednášce jsme uvedli tuto implementaci funkce `scale-list` a upozornili na to, že podle našich dosavadních znalostí by vlastně neměla fungovat.

```
(defun scale-list (list factor)
  (labels ((prod (x) (* x factor)))
    (mapcar #'prod list)))
```

Na této přednášce se zaměříme na to, proč tomu tak je, v čem je problém a co je potřeba udělat (a co je v Lispu uděláno) k jeho řešení. Začneme tím, že si zopakujeme, co to je uživatelsky definovaná funkce.

Víme, že funkce jsou hodnoty (to jsme se dozvěděli minule), které lze *aplikovat* na jiné hodnoty. Uživatelsky definované funkce se definují speciálním operátorem `defun` a od minule také speciálním operátorem `labels`. Při definici funkce se zadává její

- jméno
- seznam parametrů
- tělo

Funkci samotnou můžeme chápat jako datovou strukturu, která obsahuje z těchto informací druhou a třetí, tedy seznam parametrů (na minulé přednášce byla ukázána jeho nová podoba s klíčovým slovem `&rest`) a tělo, což je symbolický výraz.

Svůj název si funkce pamatovat nemusí (později na této přednášce zavedeme i funkce, které žádný název nemají); funkce je ale uložena jako hodnota funkční vazby svého názvu v nějakém prostředí.

Aplikace uživatelsky definované funkce probíhá s využitím seznamu parametrů a těla funkce. Jak bylo vysvětleno na druhé přednášce:

Aplikace uživatelské funkce

Při aplikaci uživatelské funkce se vytvoří nové prostředí s vazbami parametrů na argumenty. Předkem tohoto prostředí se stane globální prostředí. V tomto prostředí se pak vyhodnotí tělo funkce. Nové prostředí se vytváří při každé aplikaci znovu.

Za chvíli tento popis upravíme.

2 Lexikální uzávěr

Na předchozí přednášce jsem se zmínil (poprvé u funkce `scale-list`), že některé příklady se speciálním operátorem `labels` by vlastně neměly fungovat — tedy aspoň na základě informací, které dosud máme o funkcích.

Ukážu k tomu jednoduchý příklad. Napíšeme funkci `multiplier`, která k danému číslu vytvoří funkci násobící tímto číslem:

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Vracená funkce je přesně funkce, kterou používá funkce `scale-list`.

Test:

```
> (bind a (multiplier 5))
#<LOCAL-FUNCTION PROD subfunction of MULTIPLIER>

> (bind b (multiplier 10))
#<LOCAL-FUNCTION PROD subfunction of MULTIPLIER>

> (funcall a 1)
5

> (funcall a 5)
25

> (funcall b 1)
10

> (funcall b 5)
50

> (funcall (multiplier 100) 7)
700
```

Jak vidíme, funkce `multiplier` vrátila jako svůj výsledek pokaždé jinou funkci. Funkce uložená v proměnné `a` je jistě jiná než funkce uložená v proměnné `b`, protože se různě chovají. Jak je to ale možné?

Víme, že každá uživatelská funkce obsahuje dvě informace: svůj seznam parametrů a tělo. Při aplikaci funkce na argumenty se nejprve vytvoří prostředí, ve kterém se parametry funkce navážou na (vyhodnocené) argumenty a jehož předkem je globální prostředí, a pak se tělo funkce v tomto prostředí vyhodnotí.

Podívejme se znovu na uvedený příklad, tedy funkci `multiplier`:

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Výrazy `(multiplier 5)` a `(multiplier 10)` se oba vyhodnotí na funkci. Jak bude každá z těchto funkcí vypadat?

- Seznam parametrů obou funkcí je `(x)`,
- tělo obou funkcí je `(* x factor)`.

Funkce tedy mají stejný seznam parametrů a stejné tělo. Přesto vracejí různé výsledky. (Což si my samozřejmě přejeme.)

Je to zařízeno pomocí prostředí, ve kterém se tělo funkce (při její aplikaci) vyhodnocuje. (Prezentace k přednášce na tomto místě obsahuje víc informací, které pomohou tento závěr osvětlit.)

Problém s funkcí `scale-list` z minulé přednášky byl podobný. Funkce byla definována takto:

```
(defun scale-list (list factor)
  (labels ((prod (x) (* x factor)))
    (mapcar #'prod list)))
```

Následující dva testy samozřejmě vrátí dva různé výsledky:

```
> (scale-list '(1 2 3 4) 2)
(2 4 6 8)

> (scale-list '(1 2 3 4) 10)
(10 20 30 40)
```

Lokální funkce `prod` má ovšem v obou případech stejný seznam parametrů a stejné tělo. Její aplikace uvnitř funkce `mapcar` by tedy (podle toho, co zatím o funkcích

víme) měly v obou případech vracet totéž, což samozřejmě nedělají a je to dobře. Stejně jako v minulém příkladě funkce `multiplier` je to zařízeno pomocí prostředí, ve kterém se tělo lokální funkce vyhodnocuje.

Upřesníme informaci o tom, co je výsledkem definice funkce. Výrazy, jejichž vyhodnocením vznikají nové funkce — tedy výrazy s operátorem `defun` nebo `labels` — se stejně jako všechny ostatní výrazy vždy vyhodnocují v nějakém prostředí (a to prostředí je v ten moment tzv. **aktuální**). Každý z operátorů `defun` a `labels` pak určuje prostředí, kterému říkáme **prostředí vzniku funkce** (nebo prostředí, ve kterém funkce vznikla).

- U operátoru `defun` je to prostředí, ve kterém byl výraz s operátorem `defun` vyhodnocen (obvykle je to tedy globální prostředí), tj. **aktuální prostředí**.
- U operátoru `labels` je to **prostředí tímto operátorem vytvořené** (jehož je, jak víme, aktuální prostředí předkem).

Pokud se tedy vrátíme k naší definici funkce `multiplier`:

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

pak prostředí vzniku funkce `multiplier` je prostředí, ve kterém byl tento `defun`-výraz vyhodnocen. To je obvykle **globální prostředí**, protože výrazy s operátorem `defun` obvykle vyhodnocujemeve zdrojovém souboru, jak jsme zvyklí.

U lokální funkce `prod` v tomto příkladě je prostředím vzniku prostředí vytvořené speciálním operátorem `labels`. Když například aplikujeme funkci `multiplier` na hodnotu `5`, tj. když vyhodnotíme výraz `(multiplier 5)`, bude to následující prostředí:

Globální prostředí

symbol	hodnota	typ vazby
`multiplier`	`#<FUNCTION ...>`	*funkční*
`pi`	`3.141...`	*hodnotová*
⋮	⋮	⋮

↑

Prostředí funkce `multiplier`

symbol	hodnota	typ vazby
`factor`	5	*hodnotová*

↑

Prostředí operátoru `labels`

symbol	hodnota	typ vazby
`prod`	`#<LOCAL-FUNCTION PROD ... >`	*funkční*

Pro zopakování: každá funkce obsahuje následující informace

- seznam parametrů,
- tělo,
- prostředí vzniku.

Díky tomu můžeme vylepšit naše pravidla aplikace uživatelské funkce:

Aplikace uživatelské funkce

Při aplikaci uživatelské funkce se vytvoří nové prostředí s vazbami parametrů na argumenty. Předkem tohoto prostředí se stane prostředí vzniku funkce. V tomto prostředí se pak vyhodnotí tělo. Nové prostředí se vytváří při každé aplikaci znovu.

Funkce, která nese informaci o prostředí svého vzniku (v Lispu je to každá), se nazývá *lexikální uzávěr* (stručně *uzávěr*).

Na prezentaci z přednášky je na tomto místě rozebraný příklad, který nám pomůže upravená pravidla pochopit a který ukazuje, že vedou k očekávaným výsledkům.

3 Anonymní funkce

Jak už jsem řekl, ne každá funkce musí mít název (být pojmenovaná). Funkce, které ho nemají, se nazývají *anonymní*. Takové funkce se nevytvářejí operátory `defun` nebo `labels`, protože ty vytvářejí funkce pojmenované.

K vytváření funkcí anonymních slouží speciální operátor `lambda`. Například vyhodnocením výrazu

```
(lambda (b h) (* 1/2 b h))
```

vznikne funkce, která počítá obsah trojúhelníka s danou základnou a výškou:

```
> (funcall (lambda (b h) (* 1/2 b h))
           5
           6)
15
```

Složený výraz, jehož prvním prvkem je symbol `lambda` se nazývá *λ-výraz*. Symbol `lambda` je speciální operátor, výraz musí mít kromě symbolu `lambda` další dva prvkyt: *seznam parametrů*, což je seznam ve tvaru, který už známe, a *tělo* vytvářené funkce.

```
    seznam parametrů
(lambda (b h) (* 1/2 b h))
                 tělo funkce
```

Vyhodnocením λ-výrazu vznikne nová funkce, jejíž seznam parametrů a tělo budou dány argumenty λ-výrazu. Prostředím vzniku nové funkce bude prostředí, ve kterém byl λ-výraz vyhodnocen.

Na začátku přednášky jsme si ukazovali funkci `sum-sequence` na sčítání prvků posloupnosti. Takto můžeme s pomocí anonymní funkce sečíst členy posloupnosti druhých mocnin přirozených čísel:

```
> (sum-sequence (lambda (x) (expt x 2)) 1 100)
338350
```

Pomocí operátoru `lambda` můžeme zjednodušit naši funkci `multiplier`. Funkce původně vracela funkci vzniklou jako lokální funkce `prod` pomocí speciálního operátoru `labels`.

```
(defun multiplier (factor)
  (labels ((prod (x)
             (* x factor)))
    #'prod))
```

Název funkce `prod` ale není k ničemu potřeba, můžeme ji vytvořit jako funkci anonymní:

```
(defun multiplier (factor)
  (lambda (x) (* x factor)))
```

Výsledkem musí být (a je) lexikální uzávěr, protože funkce si musí pamatovat vazbu symbolu `add` ze svého prostředí vzniku.

A konečně, funkce `scale-list` byla na minulé přednášce definována takto:

```
(defun scale-list (list factor)
  (labels ((prod (x) (* x factor)))
    (mapcar #'prod list)))
```

Takto ji můžeme přepsat pomocí anonymní funkce:

```
(defun scale-list (list factor)
  (mapcar (lambda (x) (* x factor))
          list))
```

nebo pomocí funkce `multiplier`:

```
(defun scale-list (list factor)
  (mapcar (multiplier factor)
          list))
```

Opět, výraz `(lambda (x) (* x factor))` musí vracet lexikální uzávěr, protože vrácená funkce si musí pamatovat prostředí, ve kterém vznikla, aby symbol `factor` měl pokaždé správnou hodnotu.

4 Posloupnosti

Počínaje touto částí až do konce přednášky probereme jednu důležitou aplikaci lexikálních uzávěrů a funkcí vyššího řádu. Ukážeme si, jak pomocí funkcí (lexikálních uzávěrů) jako hodnot můžeme zadávat nekonečně velké objekty, například matematické posloupnosti.

První příklad se bude týkat **sčítání prvků posloupnosti**. V minulosti jsme už psali funkci podobnou této:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ from (sum-numbers (+ from 1) to))))
```

Funkce sečte všechna celá čísla počínaje prvním a konče druhým argumentem (neboli celá čísla ze zadaného intervalu):

```
> (sum-numbers 0 100)
5050
```

Funkce sama o sobě není užitečná, protože prvky aritmetické posloupnosti můžeme sečíst pomocí vzorečku. Poslouží nám ale jako základ k dalším úvahám.

Co kdybychom chtěli sečíst druhé mocniny přirozených čísel ze zadaného intervalu? Funkce, která to dělá, by mohla vypadat například takto:

```
(defun sum-squares (from to)
  (if (< to from)
      0
    (+ (expt from 2) (sum-squares (1+ from) to))))
```

Funkce sčítá členy *posloupnosti druhých mocnin přirozených čísel* (včetně nuly). Je to posloupnost, jejíž člen o indexu n je číslo n^2, tedy posloupnost

$$0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121 \ldots$$

(člen o indexu 0 je 0, další 1, $2^2 = 4$, $3^2 = 9$, atd.; posloupnosti budeme indexovat od nuly)

Například součet členů počínaje třetím a konče šestým je $9 + 16 + 25 + 36 = 86$, jak můžeme ověřit aplikací funkce:

```
> (sum-squares 3 6)
86
```

Funkci můžeme přepsat pomocí funkce `power2`, kterou jsme si kdysi napsali,

```
(defun power2 (x)
  (* x x))
```

takto:

```
(defun sum-squares (from to)
  (if (< to from)
      0
    (+ (power2 from) (sum-squares (1+ from) to))))
```

Obecně, součet členů posloupnosti čísel je jednou z nejvíce v praxi řešených matematických úloh.

Z našeho pohledu může být posloupnost dána funkcí, která pro dané přirozené číslo n vrátí jako hodnotu n-tý člen posloupnosti. Například posloupnost druhých mocnin přirozených čísel je dána (říkáme také *reprezentována*) funkcí `power2`.

Zdrojový kód funkce `sum-numbers` a `sum-squares` se od sebe liší jen velmi málo. To ještě vynikne, pokud v první funkci použijeme funkci `identity`:

```
(defun identity (x)
  x)
```

Funkci `sum-numbers` můžeme s její pomocí přepsat:

```
(defun sum-numbers (from to)
  (if (< to from)
      0
    (+ (identity from) (sum-numbers (+ from 1) to))))
```

Posloupnost přirozených čísel včetně nuly, tedy posloupnost

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, \ldots$$

totiž lze zadat (reprezentovat) funkcí `identity`. Vidíme, že i takové triviální funkce jako `identity` mohou být užitečné. Dnes se setkáme ještě s jednou takovou.

Dalším příkladem posloupnosti reprezentované funkcí je už známá Fibonacciho posloupnost zadaná funkcí

```
(defun fib (n)
  (cond ((= n 0) 0)
        ((= n 1) 1)
        (t (+ (fib (- n 2)) (fib (- n 1))))))
```

(umíme zrychlit) Samozřejmě můžeme vymyslet mnoho dalších funkcí, které reprezentují nějakou zajímavou posloupnost.

Pro přístup k prvkům posloupnosti si napíšeme funkci `mem` (jako *member*). Vypadá to jako formalita, ale funkce nám pomůže zlepšit čitelnost kódu a případné budoucí změny v reprezentaci posloupností.

```
(defun mem (seq index)
  (funcall seq index))
```

Následující obecná funkce sčítá členy dané posloupnosti v zadaném intervalu. Posloupnost je dána funkcí, která ji reprezentuje (parametr `seq`):

```
(defun sum-sequence (seq from to)
  (if (< to from)
      0
    (+ (mem seq from)
       (sum-sequence seq (+ from 1) to))))
```

Součet čísel v daném intervalu teď můžeme vypočítat takto:

```
> (sum-sequence #'identity 0 100)
5050
```

Součet druhých mocnin takto:

```
> (sum-sequence #'power2 0 100)
338350
```

Funkce `sum-sequence` je samozřejmě dalším příkladem funkce vyššího řádu.

5 Dynamické vytváření nových funkcí

Jak jsme si řekli minule, funkcí vyššího řádu rozumíme funkci, která je určena k práci s jinými funkcemi. Konkrétně může funkce přijímat jako argumenty, nebo je vracet jako výsledek. Dosud jsme se zabývali převážně prvním případem, jedinou výjimkou byla funkce `multiplier`. Nyní se podíváme podrobněji na případ druhý a předvedeme si několik funkcí vyššího řádu, které **dynamicky** (tj. za běhu programu) vytvářejí funkce nové. K tomu často používají operátor `lambda`.

Příklady se budou opět týkat posloupností. Abychom s nimi mohli snadno pracovat, napíšeme si nejprve funkci, která vytvoří seznam obsahující zadaný počet prvních členů dané posloupnosti:

```
(defun seq-to-list (seq len)
  (labels ((stl (index)
             (if (>= index len)
                 '()
               (cons (mem seq index)
                     (stl (+ index 1))))))
    (stl 0)))
```

Funkci můžeme otestovat například na posloupnosti čtverců:

```
> (seq-to-list (lambda (x) (* x x)) 20)
(0 1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256 289 324 361)
```

nebo na Fibonacciho posloupnosti:

```
> (seq-to-list #'fib 30)
(0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
6765 10946 17711 28657 46368 75025 121393 196418 317811 514229)
```

Funkce `seq-to-list` je samozřejmě funkce vyššího řádu, protože jako svůj argument očekává funkci.

V této části už dále nebudeme ukazovat, co funkce `seq-to-list` kdy vrátí, abychom zbytečně neplýtvali místem. Ale kdykoliv a pro libovolný příklad posloupnosti, o kterém bude dále řeč, si to můžete sami vyzkoušet.

Teď bude následovat několik příkladů, jak lze pomocí anonymních funkcí s posloupnostmi pracovat.

Posloupnost, jejíž všechny členy jsou stejné (tzv. *konstantní posloupnost*) můžeme pomocí funkce zadat snadno. Například posloupnost, jejíž všechny členy jsou 10, zadáme funkcí

```
(defun constantly-10 (n)
  10)
```

Tak můžeme zadat konstantní posloupnost s libovolnou hodnotou jejích členů. To je ovšem trochu nešikovné. Podíváme se tedy po nějaké abstrakci, která umožní zadávat konstantní posloupnosti jednodušeji.

Napíšeme funkci, která požadovanou konstantní posloupnost vrátí jako svou hodnotu:

```
(defun constantly (c)
  (lambda (n) c))
```

Otestovat ji můžete například vyhodnocením výrazu

```
(seq-to-list (constantly 5) 20)
```

Součtem dvou posloupností rozumíme posloupnost, jejíž členy jsou součty příslušných členů původních posloupností:

```
(defun seq-+ (seq1 seq2)
  (lambda (n)
    (+ (mem seq1 n) (mem seq2 n))))
```

Posloupnost, jejíž člen o indexu n je dán předpisem

$$a_n = n^2 + 1,$$

je tedy reprezentována funkcí vzniklou vyhodnocením výrazu

```
(seq-+ #'power2 (constantly 1))
```

Uvědomte si prosím znovu, že funkce, kterou vrací funkce `seq-+` jako výsledek, musí být lexikální uzávěr. Musí si totiž pamatovat aktuální vazby symbolů `seq1` a `seq2` v době, kdy byla funkce `seq-+` aplikována. Pokud vám to není jasné, znázorněte si přesně prostředí, která se použijí při aplikaci poslední uvedené posloupnosti na nějaké číslo, třeba

```
(mem (seq-+ #'power2 (constantly 1)) 10)
```

Obecná operace se dvěma posloupnostmi:

```
(defun seq-op (op seq1 seq2)
  (lambda (n)
    (funcall op (mem seq1 n) (mem seq2 n))))
```

Aritmetický průměr dvou posloupností:

```
(seq-op (lambda (x y) (/ (+ x y) 2)) seq1 seq2)
```

Hledání v posloupnosti. Funkce `first-index` vrací index prvního členu posloupnosti, který splňuje zadanou podmínku:

```
(defun first-index (seq condition)
  (labels ((iter (index)
             (if (funcall condition (mem seq index))
                 index
               (iter (1+ index)))))
    (iter 0)))
```

(Funkce se zacyklí, pokud žádný člen posloupnosti podmínku nesplňuje.)

Kolikáté Fibonacciho číslo je větší než 100?

```
(first-index #'fib (lambda (x) (> x 100)))
```

Kolikáté Fibonacciho číslo je dělitelné 14 (kromě nuly)?

```
(first-index #'fib (lambda (x)
                     (and (> x 0)
                          (= (rem x 14) 0))))
```

6 Příklad: bankovní účet

Vklady na účet

Vkládáme každý měsíc 10000 Kč na bankovní účet. Měsíce číslujeme od nuly, první vklad uděláme hned v měsíci č. 0 (při založení účtu). Posloupnost vkladů je dána funkcí vytvořenou výrazem

```
(constantly 10000.0)
```

Je to funkce, která pro každý měsíc vrátí 10000.0:

```
> (seq-to-list (constantly 10000.0) 17)
(10000.0 10000.0 10000.0 10000.0 10000.0 10000.0 10000.0 10000.0
10000.0 10000.0 10000.0 10000.0 10000.0 10000.0 10000.0 10000.0
10000.0)
```

Funkci si můžeme uložit do proměnné:

```
> (bind deposits (constantly 10000.0))
#<ANONYMOUS-FUNCTION subfunction of CONSTANTLY>
```

Předpokládejme, že nám banka začátkem následujícího měsíce připisuje úrok $\frac{1}{12}$ % ze zůstatku na účtu. Pro měsíc n je zůstatek B_n (B jako *balance*) na účtu tedy

$$\begin{aligned} B_0 &= d_0 \\ B_n &= B_{n-1} \cdot (1+i) + d_n \text{ pro } n > 0, \end{aligned}$$

kde

B_n: zůstatek v měsíci n
i: měsíční úrok (např. $\frac{1}{12}$ %)
d_n: vklad v měsíci n (např. 10000 Kč)

Napíšeme funkci, která z posloupnosti vkladů vytvoří posloupnost zůstatků tím, že v každém měsíci připíše úroky:

```
(defun balances (deposits interest-rate)
  (lambda (month)
    (if (= month 0)
        (mem deposits 0)
      (+ (* (mem (balances deposits interest-rate) (- month 1))
            (+ interest-rate 1))
         (mem deposits month)))))
```

Funkce samozřejmě vrací jako výsledek posloupnost (to je to, co jsme chtěli). Tohle třeba je posloupnost zůstatků v našem příkladě, když je měsíční úroková míra rovna $\frac{1}{12}$ %:

```
> (bind balances (balances deposits 1/1200))
#<ANONYMOUS-FUNCTION subfunction of BALANCES>
```

```
> (seq-to-list balances 20)
(10000.0 20008.334 30025.008 40050.03 50083.41 60125.15 70175.26
80233.74 90300.61 100375.87 110459.52 120551.58 130652.05 140760.94
150878.25 161003.99 171138.16 181280.78 191431.86 201591.39)
```

Za jak dlouho bude zůstatek alespoň milion?

```
(first-index balances (lambda (x) (>= x 1000000)))
```

Nové symboly

speciální operátor: `lambda`

Otázky a úkoly na cvičení

1. Co bude hodnotou následujícího výrazu?

```
(funcall (lambda (x) (funcall x 10)) (lambda (x) 20))
```

2. Přepište výraz

```
(let ((x a))
  tělo)
```

tak, aby neobsahoval speciální operátory `let`, `let*` a přitom se vyhodnocoval stejně.

3. Vysvětlete podrobně, jaká prostředí vznikají při aplikaci funkce `scale-list` z minulé přednášky.

4. Totéž udělejte pro obě verze funkce `scale-list` z této přednášky.

5. Mohou být následující funkce použity jako konstruktor a selektory párů?

```
(defun my-cons (x y)
  (lambda (c) (if c x y)))

(defun my-car (cons)
  (funcall cons t))

(defun my-cdr (cons)
  (funcall cons nil))
```

Vysvětlete pomocí prostředí.

6. Napište výraz, jehož hodnotou je funkce, která pro číselný argument vypočítá jeho faktoriál. Ve výrazu nesmíte použít operátory `defun` a `labels` a žádnou uživatelsky definovanou funkci.

7. Napište predikát `constant-seq-p`, který zjistí, zda prvních k zadaných členů dané posloupnosti je stejných. Funkce přijímá jako argumenty posloupnost a číslo k (abyste příklad pochopili, můžete si část posloupnosti převést na seznam funkcí `seq-to-list`):

```
> (constant-seq-p (lambda (n) (if (< n 20) 0 n))
                  20)
T

> (constant-seq-p (lambda (n) (if (< n 20) 0 n))
                  21)
NIL
```

8. Napište predikát `increasing-seq-p`, který zjistí, zda prvních k členů zadané posloupnosti roste, tj. zda každý člen je větší než předchozí. Predikát bude přijímat dva argumenty: posloupnost a číslo k.

9. Napište funkci `from-nth`, která k dané posloupnosti vytvoří novou posloupnost obsahující právě všechny členy posloupnosti původní počínaje členem o zadaném indexu. Příklad:

```
> (seq-to-list (from-nth #'identity 10) 20)
(10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29)
```

10. Napište funkci `even-members`, která vrátí posloupnost obsahující pouze členy zadané posloupnosti se sudým indexem:

```
> (seq-to-list (even-members #'identity) 20)
(0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38)

> (seq-to-list (even-members #'fib) 20)
(0 1 3 8 21 55 144 377 987 2584 6765 17711 46368 121393 317811
832040 2178309 5702887 14930352 39088169)

> (seq-to-list (even-members (lambda (n)
                               (if (evenp n) 5 10)))
               20)
(5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5)
```

11. Funkce mohou reprezentovat i jiné objekty než posloupnosti. Například funkce *`tbl`* přijímající dva argumenty může představovat tabulku čísel s 10 řádky a 10 sloupci tak, že hodnota `(funcall` *`tbl row column`*`)` je číslo na řádku s indexem *`row`* a sloupci s indexem *`column`* (číslujeme jako obvykle od nuly). Napište predikát `zero-row-p`, který přijímá dva argumenty, a to funkci a index od 0 do 9, a zjistí, zda tabulka představovaná zadanou funkcí má na řádku se zadaným indexem samé nuly.

12. Tabulku reprezentujeme funkcí stejně jako v předchozím příkladě. Napište funkci `transpose-table`, která k zadané tabulce vrátí tabulku transponovanou, tj. s vyměněnými řádky a sloupci. Například k tabulce nalevo tedy vrátí tabulku napravo:

$$\begin{array}{ccccc} 0 & 1 & 2 & \cdots & 9 \\ 0 & 2 & 4 & \cdots & 18 \\ 0 & 3 & 6 & \cdots & 27 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 10 & 20 & \cdots & 90 \end{array} \qquad \begin{array}{ccccc} 0 & 0 & 0 & \cdots & 0 \\ 1 & 2 & 3 & \cdots & 10 \\ 2 & 4 & 6 & \cdots & 20 \\ \vdots & \vdots & \vdots & & \vdots \\ 9 & 18 & 27 & \cdots & 90 \end{array}$$